3. ITERATIVE DEEPNING SEARCH

```
def depth_limited_search(node, goal, graph, limit, path):
    if node == goal:
        return path + [node]

    if limit == 0:
        return None

    for child in graph.get(node, []):
        if child not in path:
            result = depth_limited_search(
                child, goal, graph, limit - 1, path + [node]
            )
            if result is not None:
                return result

    return None


def iterative_deepening_search(initial, goal, graph, maximum_depth):
    for depth in range(maximum_depth + 1):
        result = depth_limited_search(
            initial, goal, graph, depth, []
        )

        if result is not None:
            print("Goal found!")
            print("Solution Path:", " -> ".join(result))
            print("Depth:", len(result) - 1)
            return
```

```
    print("Goal not found within the specified maximum depth.")


# Input
initial = input("Enter initial state: ").strip()
goal = input("Enter goal state: ").strip()
maximum_depth = int(input("Enter maximum depth: "))

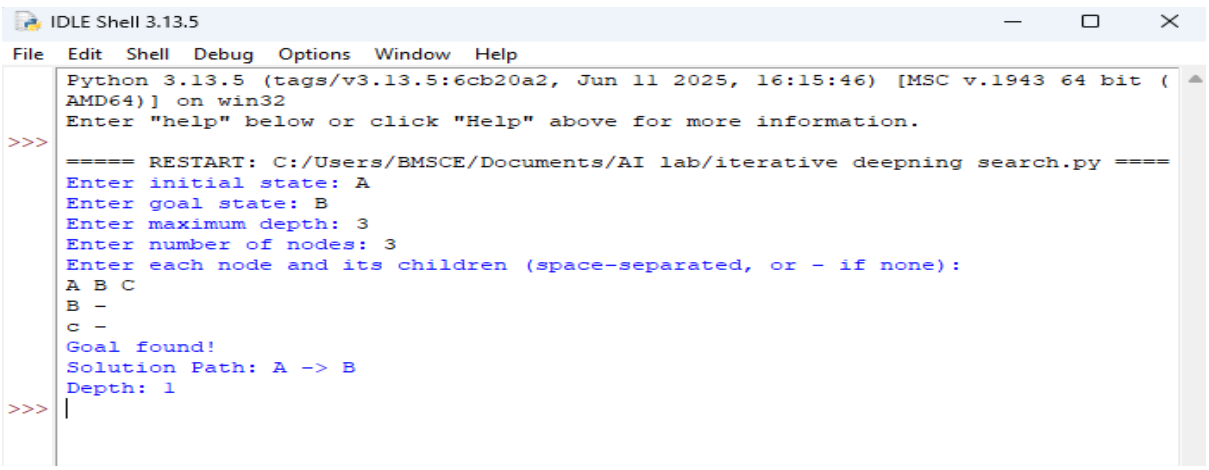
n = int(input("Enter number of nodes: "))
graph = {}

print("Enter each node and its children (space-separated, or - if none):")
for _ in range(n):
    data = input().strip().split()
    node = data[0]
    children = [] if len(data) == 1 or data[1] == "-" else data[1:]
    graph[node] = children

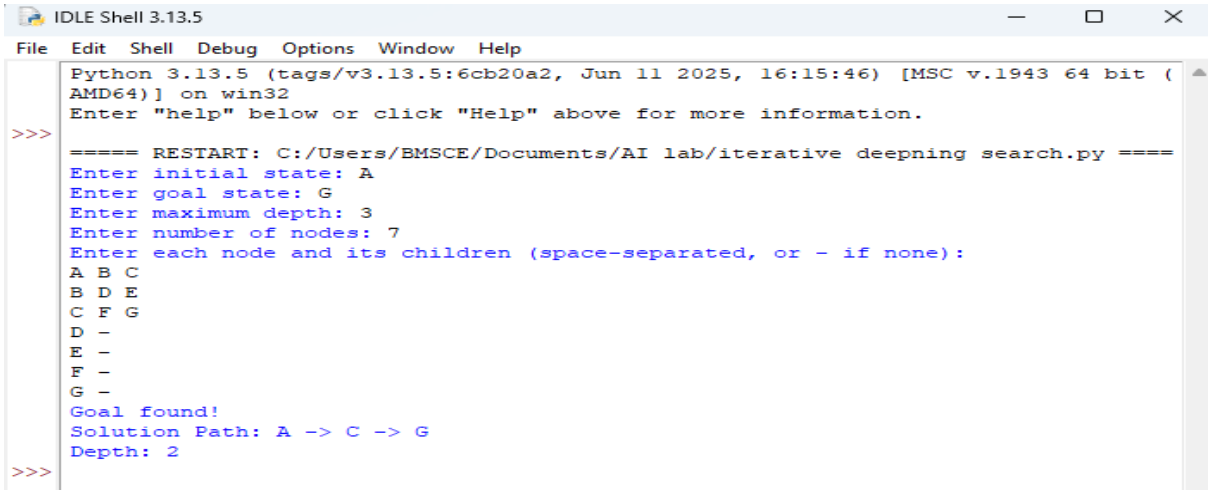
# Perform Iterative Deepening Search
iterative_deepening_search(initial, goal, graph, maximum_depth)
```

Output:

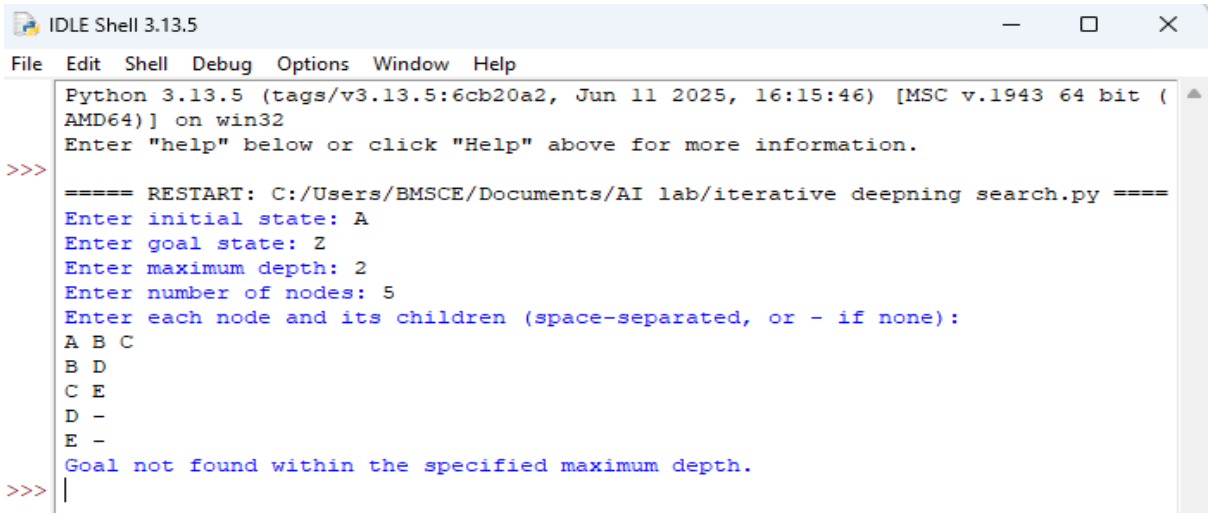
Test case 1 : Goal at depth 1



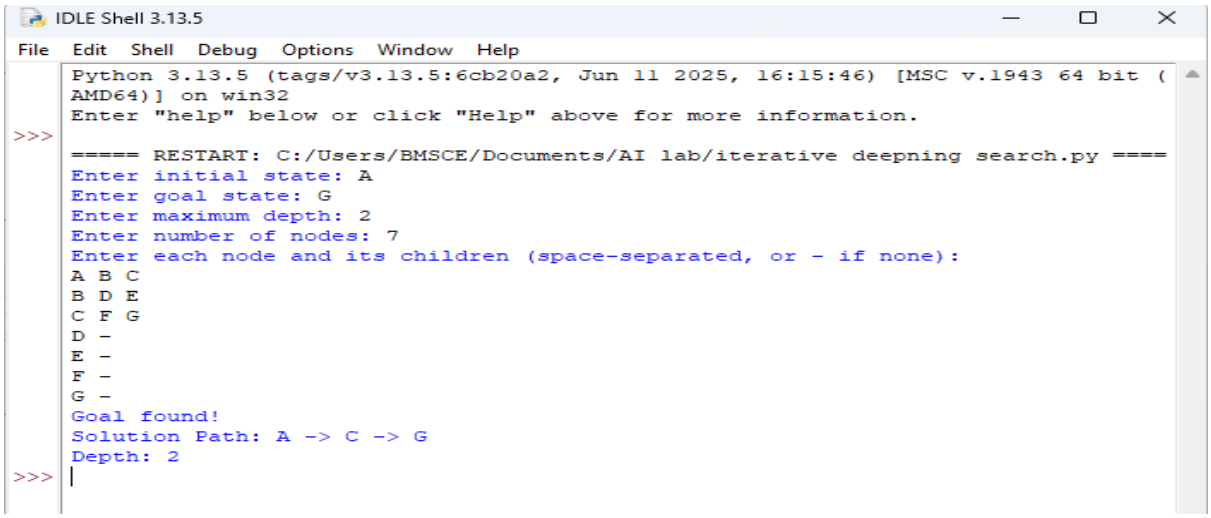
Test case 2 : Goal at depth 2



Test case 3 : Goal not present in the search tree



Test case 4 : Goal at maximum depth



Test case 5: Goal not present in the search tree

